Zadanie nr 4 - Sieć samoorganizująca się SOM w kompresji obrazów

Inteligentna Analiza Danych

Jędrzej Kostyk 264339

2 listopada 2026

1 Cel zadania

Celem zadania było zaimplementowanie jednowarstwowej liniowej sieci samoorganizującej się (SOM), trenowanej bez nadzoru ze współzawodnictwem neuronów, i użycie jej do kompresji obrazów w skali szarości. Dla ramki 2×2 piksele dobrano parametry nauki, a następnie zbadano, jak początkowa liczba neuronów (4–256) wpływa na współczynnik kompresji CR i jakość obrazu odtworzonego PSNR, oraz wykreślono zależność CR od PSNR. Dodatkowo sprawdzono wpływ rozmiaru ramki i sposobu inicjalizacji wag.

2 Wstęp teoretyczny

Sieć SOM i trening ze współzawodnictwem. Sieć ma architekturę taką jak MADALINE: jedną warstwę K neuronów liniowych, które liczą iloczyny skalarne $y_i = \mathbf{w}_i \cdot \mathbf{x}$ znormalizowanego wejścia ze swoimi znormalizowanymi wagami [3, 2]. Wzorce treningowe zawierają tylko wejścia (uczenie nienadzorowane). Po podaniu wzorca wybierany jest neuron zwycięski $m = \arg\max_i y_i$ i tylko jego wagi są modyfikowane według reguły „zwycięzca bierze wszystko" (WTA, Algorytm 2.1 w [3]):

$$\mathbf{w}_m \leftarrow \frac{\mathbf{w}_m + \eta\,(\mathbf{x} - \mathbf{w}_m)}{\|\mathbf{w}_m + \eta\,(\mathbf{x} - \mathbf{w}_m)\|}. \tag{1}$$

Wagi zwycięzcy przesuwają się w stronę wzorca, więc po treningu każdy żywy neuron reprezentuje „średni" kierunek jednego skupiska wektorów wejściowych. Neurony, które nigdy nie wygrały, nazywa się *martwymi* i usuwa się je z sieci.

Kompresja obrazu (Algorytm 2.2 w [3]). Obraz $N \times N$ dzieli się na $(N/n)^2$ ramek $n \times n$, które po spłaszczeniu i normalizacji są wzorcami treningowymi. Po treningu i usunięciu martwych neuronów każda ramka $\mathbf{x}_k$ jest kodowana dwiema liczbami całkowitymi: indeksem neuronu zwycięskiego $\mathcal{L}_k$ (kierunek, czyli „kształt" ramki) oraz skwantowaną długością $\mathcal{T}_k = \mathrm{round}(\|\mathbf{x}_k\|/q)$ (jasność ramki), gdzie q to współczynnik kwantyzacji. Do pliku trafiają też wagi wszystkich K żywych neuronów $\mathcal{W}$. Dekodowanie odtwarza ramkę jako $\mathbf{x}'_k = q\,\mathcal{T}_k\,\mathcal{W}_{\mathcal{L}_k}$. Jest to w istocie kwantyzacja wektorowa [1]: zamiast pikseli zapamiętuje się numer „wzorcowej" ramki ze słownika.

Miary efektywności. Jakość obrazu odtworzonego mierzy szczytowy stosunek sygnału do szumu

$$\mathrm{PSNR} = 10 \log_{10} \frac{255^2}{\mathrm{MSE}}\ [\mathrm{dB}], \qquad \mathrm{MSE} = \frac{1}{N^2} \sum_{i,j} (x_{ij} - x'_{ij})^2, \tag{2}$$

a stopień kompresji to iloraz liczby bitów obrazu oryginalnego i skompresowanego (wzór 2.4 w [3]):

$$\mathrm{CR} = \frac{8N^2}{N_{\mathcal{W}} + N_{\mathcal{L}} + N_{\mathcal{T}}}, \tag{3}$$

$$N_{\mathcal{W}} = 8Kn^2, \qquad N_{\mathcal{L}} = \Big(\frac{N}{n}\Big)^2 \big(\lfloor \log_2(K-1) \rfloor + 1\big),$$
$$N_{\mathcal{T}} = \Big(\frac{N}{n}\Big)^2 \big(\lfloor \log_2 \max_k \mathcal{T}_k \rfloor + 1\big).$$

Składniki mianownika to liczby bitów potrzebne na wagi (8 bitów na wagę), indeksy neuronów zwycięskich i skwantowane długości ramek.

Ograniczenie CR dla ramki 2×2. Ramka oryginalna zajmuje $8\cdot4 = 32$ bity. Po kompresji potrzeba co najmniej 1 bitu na indeks, a przy $q = 10$ największa długość ramki ($2 \cdot 255 = 510$) daje $\mathcal{T} \le 51$, czyli 6 bitów. Dlatego nawet przy jednym neuronie $\mathrm{CR} \le 32/7 \approx 4{,}57$. Każde podwojenie liczby żywych neuronów dokłada 1 bit na ramkę.

Szczegóły implementacji. Program `som.py` realizuje Algorytmy 2.1 i 2.2. Wagi początkowe są losowane z $[-1, 1]$ i normalizowane, a kolejność ramek jest losowana w każdej epoce. Neuron jest uznawany za martwy, gdy jego wagi nie zostały zmienione ani razu. Wagi zapisane w „pliku" kwantujemy do 8 bitów, zgodnie z założeniem wzoru (3), a obraz odtworzony zaokrąglamy do liczb całkowitych z $[0, 255]$. W przykładowym raporcie w materiałach do wykładu użyto 5 bitów na wagę, a nie 8 jak we wzorze (3). Liczba bitów na wagę jest więc parametrem programu (`--weight-bits`, domyślnie 8). Przy ramce 2×2 wagi stanowią tylko 1–2% rozmiaru pliku, więc wybór 5 albo 8 bitów zmienia CR o mniej niż 1%.

3 Eksperymenty i wyniki

Wspólne założenia wszystkich eksperymentów:

- obrazy `zelda`, `cameraman` i `peppers` z materiałów do kursu, 128×128 pikseli, 8-bitowa skala szarości (dostarczone jako pliki PNG),
- ramka 2×2, czyli 4096 ramek i wektory o 4 elementach (o ile nie podano inaczej),
- parametry domyślne: 20 epok, $\eta = 0{,}2$, $q = 10$, 8 bitów na wagę,
- każdy wariant uruchomiono z 5 ziarnami generatora (różne wagi początkowe i kolejność ramek), podano średnie, a w tabelach parametrów także odchylenia standardowe PSNR.

Przykładowy raport programu (`python som.py images/zelda.png --neurons 64`):

```
resolution = 128 x 128
frame_size = 2 x 2

num_frames = 4096
num_neurons (start) = 64
num_neurons = 11
num_weights_per_neuron = 4
max_quantized_length = 34

bits_per_index = 4
bits_per_length = 6
bits_per_weight = 8

org_size_bits = 131072
cmp_size_bits = 41312

w_1 = [ 0.373 0.404 0.506 0.671]
w_2 = [ 0.420 0.537 0.467 0.561]
w_3 = [ 0.624 0.357 0.631 0.302]
w_4 = [ 0.741 0.420 0.388 0.349]
CR = 3.173
PSNR = 31.789 dB
```

3.1 Eksperyment nr 1 – dobór parametrów nauki

3.1.1 Założenia

Obraz `zelda`, ramka 2×2, $K = 64$ neurony. Zmieniano po jednym parametrze, a pozostałe miały wartości sugerowane w specyfikacji.

3.1.2 Przebieg

Zbadano $\eta \in \{0{,}01; \ldots; 0{,}9\}$, liczbę epok od 1 do 50 oraz q od 1 do 40. Dla każdej wartości wykonano 5 treningów.

3.1.3 Rezultat

Wyniki są w Tab. 1.

- **Współczynnik uczenia:** wartości 0,05–0,2 dają podobny PSNR (ok. 32 dB, różnice mniejsze niż rozrzut między uruchomieniami). Zbyt małe $\eta = 0{,}01$ nie pozwala wagom dojść do środków skupisk w 20 epokach. Zbyt duże (0,5; 0,9) obniża PSNR i zwiększa rozrzut, bo wagi zwycięzcy „skaczą” za każdą kolejną ramką zamiast uśredniać skupisko.

Tabela 1: Wpływ parametrów nauki na PSNR i CR (`zelda`, $K = 64$, średnia z 5 uruchomień)

η	PSNR [dB]	CR	żywe
0,01	$31{,}05 \pm 0{,}37$	3,172	11,4
0,05	$31{,}93 \pm 0{,}24$	3,168	12,8
0,1	$32{,}27 \pm 0{,}13$	3,169	12,6
0,2	$31{,}94 \pm 0{,}52$	3,167	13,2
0,5	$31{,}14 \pm 0{,}74$	3,166	13,6
0,9	$29{,}72 \pm 1{,}29$	3,166	13,8

epoki	PSNR [dB]	CR	żywe
1	$30{,}62 \pm 0{,}35$	3,244	10,4
2	$30{,}88 \pm 0{,}42$	3,171	11,6
5	$31{,}77 \pm 0{,}24$	3,170	12,2
10	$31{,}98 \pm 0{,}65$	3,168	12,8
20	$31{,}94 \pm 0{,}52$	3,167	13,2
50	$32{,}41 \pm 0{,}30$	3,167	13,4

q	PSNR [dB]	CR	żywe
1	$32{,}17 \pm 0{,}55$	2,442	13,2
2	$32{,}16 \pm 0{,}55$	2,644	13,2
5	$32{,}12 \pm 0{,}54$	2,882	13,2
10	$31{,}94 \pm 0{,}52$	3,167	13,2
20	$31{,}33 \pm 0{,}45$	3,515	13,2
40	$29{,}48 \pm 0{,}30$	3,949	13,2

- **Liczba epok:** już po 5 epokach PSNR jest bliski wartości końcowej. Dalsze epoki dają ok. 0,5 dB, więc sugerowane 20 epok jest wystarczające.
- **Współczynnik kwantyzacji q:** nie wpływa na trening, tylko na zapis jasności ramek. Większe q to mniej bitów na długość ramki, czyli wyższe CR, ale też większy błąd jasności. Przy $q = 40$ CR rośnie do 3,95, a PSNR spada o 2,5 dB. Przy $q \leq 10$ PSNR prawie się nie zmienia, bo błąd jest wtedy zdominowany przez kwantyzację kształtu ramek przez sieć.

Do dalszych eksperymentów przyjęto parametry sugerowane w specyfikacji: 20 epok, $\eta = 0{,}2$, $q = 10$.

3.2 Eksperyment nr 2 – wpływ początkowej liczby neuronów

3.2.1 Założenia

Zgodnie z punktem 4 specyfikacji: $K \in \{4, 8, 16, 32, 64, 128, 256\}$, ramka 2×2, wszystkie trzy obrazy.

3.2.2 Przebieg

Dla każdego obrazu i każdego K przeprowadzono trening, kompresję i dekodowanie (5 ziaren). Zapisano liczbę żywych neuronów, CR i PSNR.

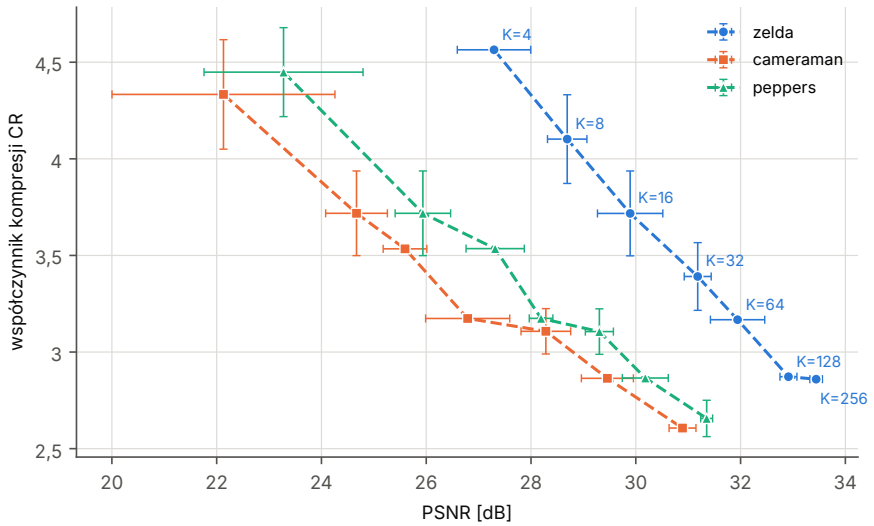
3.2.3 Rezultat

Tabela 2: Liczba żywych neuronów, CR i PSNR [dB] w zależności od początkowej liczby neuronów K

	zelda			**cameraman**			**peppers**		
$\boldsymbol{K}$	żywe	CR	PSNR	żywe	CR	PSNR	żywe	CR	PSNR
4	1,4	4,56	27,29	2,2	4,33	22,13	1,8	4,45	23,28
8	3,0	4,10	28,69	4,6	3,72	24,67	4,6	3,72	25,94
16	4,8	3,72	29,89	7,0	3,53	25,59	6,6	3,54	27,32
32	8,0	3,39	31,18	10,6	3,17	26,79	10,4	3,17	28,19
64	13,2	3,17	31,94	14,6	3,11	28,28	15,2	3,11	29,30
128	18,0	2,87	32,91	22,2	2,86	29,46	21,4	2,87	30,18
256	24,4	2,86	33,44	35,4	2,61	30,89	33,4	2,66	31,35

Charakterystyka z Rys. 1 pokazuje typowy kompromis kompresji stratnej: im więcej neuronów, tym lepsza jakość i tym słabsza kompresja. Dla `zelda` przejście od $K = 4$ do $K = 256$ podnosi PSNR z 27,3 do 33,4 dB, a CR spada z 4,56 do 2,86. `cameraman` i `peppers` mają przy tych samych K o 2–5 dB niższy PSNR, bo zawierają więcej ostrych krawędzi i drobnych szczegółów, których kilka „wzorcowych" ramek nie odtworzy (Rys. 2). `zelda` jest gładsza, a ramki gładkie są dobrze opisane już przez sam kierunek „płaski" $[\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}]$ i jasność.

Przy $K = 4$ zwykle przeżywa tylko 1–2 neurony, a CR jest bliskie ograniczeniu $32/7 \approx 4{,}57$. Sieć zapamiętuje wtedy praktycznie tylko jasność ramek, co działa jak zmniejszenie rozdzielczości obrazu dwukrotnie. Mimo to PSNR wynosi 22–27 dB, bo większość informacji w obrazie to właśnie jasność, a nie „tekstura" wewnątrz ramki 2×2.



Rysunek 1: Charakterystyka CR–PSNR dla $K = 4, \ldots, 256$ (kolejne punkty od lewej: rosnące K; słupki błędów: odchylenie standardowe z 5 uruchomień)

Neurony martwe. Najważniejsza obserwacja to bardzo duża liczba neuronów martwych (Tab. 2, Rys. 3). Z 256 neuronów przeżywa tylko 24–35, czyli ok. 9–14%. Dlatego CR i PSNR zmieniają się w funkcji K wolniej, niż wynikałoby z samej liczby neuronów. Przy przejściu od 128 do 256 neuronów dla `zelda` liczba żywych rośnie z 18 do 24, a indeks nadal mieści się na 5 bitach, więc CR praktycznie się nie zmienia (2,87 → 2,86).

Przyczyna jest geometryczna. Piksele są nieujemne, a sąsiednie piksele mają zwykle zbliżone wartości, więc wszystkie znormalizowane ramki leżą w małym obszarze sfery jednostkowej, blisko kierunku „płaskiego” $\mathbf{u} = [1, \ldots, 1]/n$. Wagi losowane z $[-1, 1]$ są natomiast rozrzucone po całej sferze. Neuron wygrywa tylko wtedy, gdy jest bliżej danych niż wszystkie inne neurony, więc większość neuronów nigdy nie wygrywa (Tab. 3).

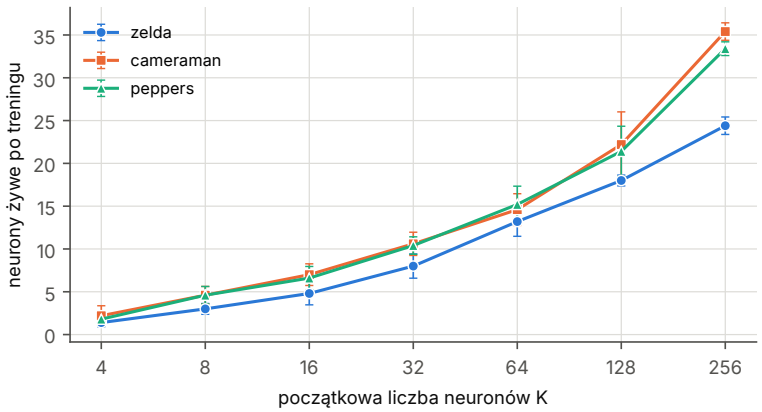
Tabela 3: Rozkład kierunków ramek obrazu `zelda`: kosinus kąta między znormalizowaną ramką a kierunkiem płaskim $\mathbf{u}$ oraz odsetek losowych wektorów z $[-1, 1]^{n^2}$, które trafiają w obszar zajęty przez dane

Ramka	**wymiar**	$\min \cos(\mathbf{x}, \mathbf{u})$	**mediana**	**losowe w obszarze**
2×2	4	0,680	0,998	10,49 %
4×4	16	0,720	0,990	0,10 %
8×8	64	0,728	0,971	0,00 %

Dla ramki 2×2 w obszar danych trafia ok. 10% losowych wektorów, co zgadza



Rysunek 2: Obrazy oryginalne i odtworzone po kompresji dla $K = 4$, 16 i 256 (ramka 2×2, ziarno 0)



Rysunek 3: Liczba neuronów, które przeżyły trening, w zależności od początkowej liczby neuronów K

się z odsetkiem żywych neuronów. Dla większych ramek ten odsetek spada

niemal do zera (eksperyment nr 3).

3.3 Eksperyment nr 3 – rozmiar ramki i inicjalizacja wag (dodatkowy)

3.3.1 Założenia

Punkt 5 specyfikacji (opcjonalny). Obraz `zelda`, ramki 2×2, 4×4 i 8×8, $K = 4, \ldots, 256$. Aby sprawdzić, czy słabe wyniki dla dużych ramek wynikają z neuronów martwych, porównano też wariant, w którym wagi początkowe to losowo wybrane (znormalizowane) ramki obrazu zamiast liczb losowych z $[-1, 1]$. Reszta Algorytmu 2.1 bez zmian.

3.3.2 Przebieg

Dla każdego wariantu wykonano 5 treningów i obliczono średnie CR i PSNR.

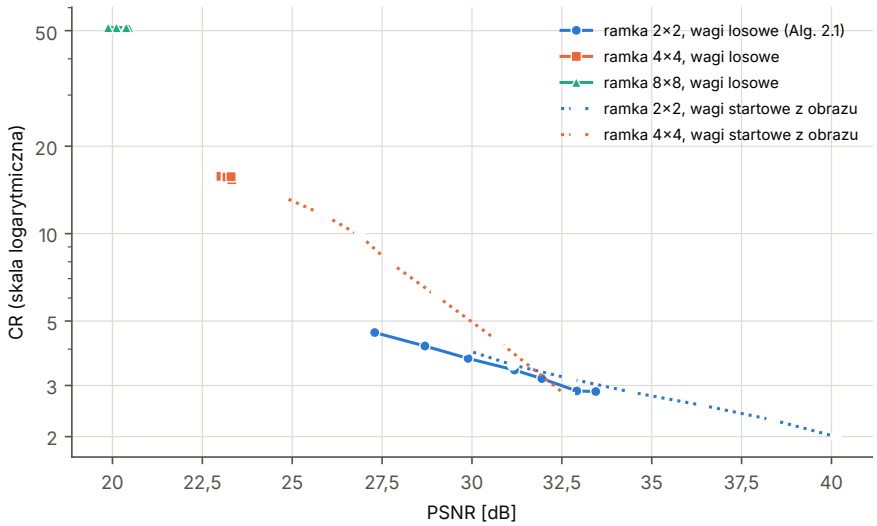
3.3.3 Rezultat

Tabela 4: CR / PSNR [dB] dla różnych rozmiarów ramki i inicjalizacji wag (`zelda`)

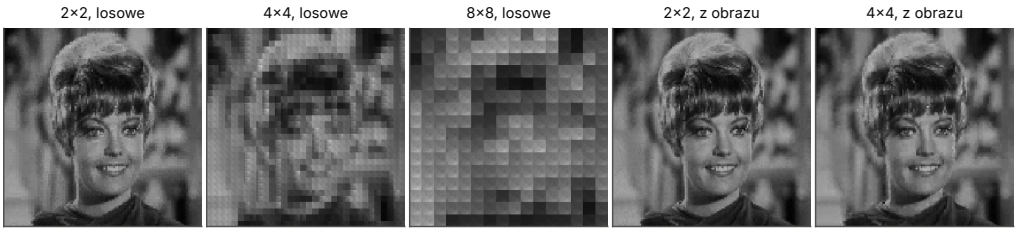
	wagi losowe z $[-1, 1]$			**wagi startowe z obrazu**	
K	2×2	4×4	8×8	2×2	4×4
16	3,7 / 29,9	15,8 / 23,0	51,2 / 19,9	3,2 / 32,8	9,8 / 26,9
64	3,2 / 31,9	15,7 / 23,2	51,2 / 20,5	2,6 / 36,4	6,1 / 29,0
256	2,9 / 33,4	15,7 / 23,3	51,2 / 20,4	2,0 / 40,1	2,7 / 32,7

Przy losowej inicjalizacji i ramkach 4×4 oraz 8×8 przeżywa niemal zawsze tylko **jeden** neuron, niezależnie od K. Sieć zapamiętuje wtedy tylko jasność każdej ramki: CR rośnie do 16 i 51, ale PSNR spada do 23 i 20 dB, a na obrazie widać wyraźne bloki (Rys. 5). Zwiększanie K nic nie daje, bo w wymiarze 16 czy 64 losowy wektor niemal nigdy nie trafia w obszar danych (Tab. 3).

Gdy wagi startowe są ramkami obrazu, wszystkie neurony przeżywają, a charakterystyka znacznie się poszerza (Rys. 4). Dla ramki 2×2 i $K = 256$ PSNR rośnie do 40,1 dB (przy CR = 2,0), a ramka 4×4 daje płynny zakres od CR $\approx$ 13,5 (PSNR 24,7 dB) do CR $\approx$ 2,7 (PSNR 32,7 dB). Większa ramka pozwala więc osiągnąć wyższe CR, ale pod warunkiem, że neurony faktycznie się uczą. Przy dużych ramkach i dużym K rośnie też udział wag w pliku: dla 8×8 i 256 neuronów same wagi zajmowałyby tyle bitów, co cały obraz oryginalny.



Rysunek 4: Charakterystyki CR–PSNR dla różnych rozmiarów ramki i inicjalizacji wag (`zelda`, $K = 4, \ldots, 256$)



Rysunek 5: Obraz `zelda` odtworzony dla $K = 256$ przy różnych ramkach i inicjalizacjach

4 Wnioski

1. Zaimplementowana sieć SOM trenowana regułą WTA (Algorytm 2.1) i schemat kompresji z Algorytmu 2.2 działają poprawnie. Obraz odtworzony z list $\mathcal{L}$, $\mathcal{T}$, $\mathcal{W}$ jest dobrze rozpoznawalny już przy kilku neuronach, a przy ramce 2×2 uzyskano CR od ok. 2,6 do 4,6 przy PSNR od 22 do 33 dB, zależnie od obrazu i liczby neuronów.

2. Liczba neuronów steruje kompromisem jakość–kompresja: więcej neuronów to więcej „wzorcowych” ramek, wyższy PSNR i więcej bitów na indeks, czyli niższe CR. Obrazy z drobnymi szczegółami (`cameraman`, `peppers`) kompresują się gorzej niż gładki obraz `zelda`.

3. Parametry sugerowane w specyfikacji (20 epok, $\eta = 0{,}2$, $q = 10$) są

rozsądne: PSNR słabo zależy od η w zakresie 0,05–0,2 i od liczby epok powyżej 5. Współczynnik q to dodatkowe pokrętło kompromisu: większe q daje wyższe CR kosztem jakości.

4. Dla ramki 2×2 CR nie może przekroczyć ok. 4,57, bo każda ramka wymaga co najmniej 7 bitów zamiast 32. Wyższe CR wymaga większych ramek.

5. Głównym ograniczeniem metody są neurony martwe. Ramki obrazu zajmują mały obszar sfery jednostkowej, a wagi losowane z $[-1, 1]$ trafiają tam rzadko: przeżywa ok. 10% neuronów dla ramek 2×2 i praktycznie tylko jeden dla ramek 4×4 i 8×8. Inicjalizacja wag ramkami obrazu usuwa ten problem i wyraźnie poprawia charakterystykę CR–PSNR.

5 Załączniki

Pełny kod znajduje się w plikach `som.py`, `eksperymenty.py` i `wykresy.py`. Poniżej trening sieci (Algorytm 2.1) i rdzeń kompresji (Algorytm 2.2) z pliku `som.py`:

```
def train(self, X, epochs, eta, rng, shuffle=True):
    X = X[np.linalg.norm(X, axis=1) > 0]
    order = np.arange(len(X))
    for _ in range(epochs):
        if shuffle:
            rng.shuffle(order)
        for k in order:
            x = X[k]
            m = int(np.argmax(self.W @ x)) # neuron zwycieski
            w = self.W[m] + eta * (x - self.W[m]) # w_m = w_m + eta (x -
                w_m)
            self.W[m] = w / np.linalg.norm(w) # normalizacja
            self.updated[m] = True

# compress():
frames = image_to_frames(img, frame) # ramki n x n -> wektory n^2
Xn, lengths = normalize_rows(frames)
som = SOM(neurons, frame * frame, rng)
som.train(Xn, epochs, eta, rng, shuffle)
K = som.remove_dead() # usuniecie neuronow martwych
L = som.winners(Xn) # lista L - indeksy zwyciezcow
T = np.rint(lengths / q).astype(int) # lista T - dlugosci / q
W = quantize_weights(som.W, weight_bits) # lista W - wagi (8 bitow)
decoded = frames_to_image(q * T[:, None] * W[L], frame, img.shape)
```

Bibliografia

[1] Robert M. Gray. Vector quantization. *IEEE ASSP Magazine*, 1(2):4–29, 1984.

[2] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[3] Materiały do Wykładu nr 3, Rozdziały 1–2 – Trening nienadzorowany i sieć samoorganizująca się SOM. Kurs Inteligentna Analiza Danych, Instytut Informatyki Politechniki Łódzkiej.